UPES

UNIVERSITY OF TOMORROW

Finance Credit Follow-Up Email Agent

Project Report

AI Enablement Internship Project Challenge

Rishiraj Singh

B.Tech CSE (CCVT)

2023-2027

1. Executive Summary

The Finance Credit Follow-Up Email Agent is an AI-inspired automation system designed to help finance teams manage overdue invoice follow-ups with minimal manual effort. Built on Python and Streamlit, the system reads pending invoice records from a CSV file, applies an intelligent tone escalation engine, generates context-appropriate follow-up emails, simulates sending, and maintains a full audit trail.

This project was developed as part of the AI Enablement Internship Project Challenge to demonstrate real-world applicability of automation and basic AI concepts in enterprise finance workflows.

2. Project Overview

2.1 Problem Statement

Finance teams in organisations routinely handle dozens of overdue invoices. Manually drafting follow-up emails that match the urgency of each overdue case is time-consuming, inconsistent, and prone to human error. There is a need for an automated system that:

- Identifies overdue invoices from structured data
- Determines the appropriate communication tone based on how overdue the invoice is
- Generates a professional, ready-to-send email
- Records all actions in an audit log for compliance and review

2.2 Objectives

- Automate the invoice follow-up email generation process
- Implement a dynamic tone escalation framework tied to overdue duration
- Provide an interactive, user-friendly dashboard via Streamlit
- Simulate email dispatch with logging for audit purposes
- Demonstrate security best practices for an AI-assisted finance tool

2.3 Scope

The current version covers CSV-based data ingestion, tone-based email generation for four escalation levels, escalation flagging for legal team cases, and a simulation-only send mode. Real SMTP integration and AI-based email personalisation are identified as future enhancements.

3. Methodology & Approach

3.1 System Architecture

The application follows a linear pipeline architecture with the following stages:

Stage	Component	Description
1	Data Ingestion	CSV file read via pandas into a DataFrame
2	Tone Engine	get_tone() maps days_overdue to escalation level
3	Email Generator	generate_email() produces structured email body
4	Send Simulation	Dry-run mode confirms action without real SMTP
5	Audit Logging	Timestamped entries written to logs.txt

3.2 Tone Escalation Logic

The core intelligence of the system lies in the tone escalation engine. Each invoice is evaluated based on its days_overdue field and assigned one of five escalation levels:

Days Overdue	Tone Level	Email Approach
1 – 7 days	Warm and Friendly	Courteous reminder; assumes oversight
8 – 14 days	Polite but Firm	Requests payment confirmation date
15 – 21 days	Formal and Serious	Demands response within 48 hours
22 – 30 days	Stern and Urgent	Final notice; warns of legal escalation
30+ days	Escalate to Legal Team	Flagged for manual review; no email sent

3.3 Email Generation

The generate_email() function accepts client details and the assigned tone, then returns a fully formatted email string using template-based logic. Using fixed templates rather than free-form AI generation ensures:

- Consistency across all communications
- Prevention of hallucinated or inaccurate financial figures
- Compliance with finance team communication standards
- Resistance to prompt injection attacks

3.4 Dashboard & UI

The Streamlit-based dashboard provides four summary metric cards (Total Invoices, Total Amount Due, Overdue Clients, Critical Cases), a full invoice table, and a per-invoice panel that displays tone level with colour coding, a Generate Email button, an inline email preview, and a Send (simulation) button.

4. Technical Stack

Layer	Technology	Purpose
Frontend UI	Streamlit	Interactive web dashboard
Backend Logic	Python 3.x	Core processing and business logic
Data Handling	Pandas	CSV ingestion and DataFrame operations
Styling	Custom CSS (via Streamlit)	Card layouts and colour coding
Logging	Python built-in file I/O	Audit trail written to logs.txt
Data Source	CSV (data.csv)	Mock invoice records
Version Control	Git / GitHub	Source code management
IDE	VS Code	Development environment

5. Project Structure

The repository follows a flat, single-directory layout suitable for a prototype-grade project:

```
finance-email-agent/   app.py            – Main application (UI + logic)
data.csv          – Mock invoice dataset   logs.txt          – Audit log
output   requirements.txt  – Python dependencies   README.md         –
Project documentation   .gitignore        – Excludes .env and sensitive files
```

6. Dataset Description

The system uses a CSV file (data.csv) containing mock invoice records with the following schema:

Field	Data Type	Description
client_name	String	Name of the client/debtor
email	String	Client email address
invoice_no	String	Unique invoice identifier (e.g. INV001)
amount	Integer	Outstanding amount in INR (₹)
due_date	Date (YYYY-MM-DD)	Original payment due date
days_overdue	Integer	Number of days past the due date

6.1 Sample Data

Client	Email	Invoice	Amount (₹)	Due Date	Days OD
Rajesh	rajesh@gmail.com	INV001	45,000	2025-04-20	5
Amit	amit@gmail.com	INV002	70,000	2025-04-10	12
Neha	neha@gmail.com	INV003	20,000	2025-03-28	18
Rohit	rohit@gmail.com	INV004	95,000	2025-03-20	27

7. Key Findings

7.1 Tone Distribution in Sample Data

Based on the four sample invoice records, the tone distribution is as follows:

- INV001 (Rajesh, 5 days overdue): Warm and Friendly
- INV002 (Amit, 12 days overdue): Polite but Firm
- INV003 (Neha, 18 days overdue): Formal and Serious
- INV004 (Rohit, 27 days overdue): Stern and Urgent

The total outstanding amount across the sample dataset is ₹2,30,000, with 75% of invoices classified as overdue beyond 7 days and 25% reaching the Stern and Urgent threshold.

7.2 Audit Log Analysis

Examination of logs.txt shows consistent timestamped entries for email generation events tied to specific invoice numbers. The log confirms the system correctly records actions per session and supports traceability for finance compliance workflows.

7.3 Template Effectiveness

Template-based email generation proved effective in ensuring factual accuracy (amounts, invoice numbers, due dates) and tone consistency. Each template maps cleanly to a distinct communication strategy without ambiguity, reducing the risk of miscommunication in high-stakes collections scenarios.

8. Security Considerations

The project implements several security mitigations appropriate for a finance-focused application:

Risk Area	Mitigation Applied
Prompt Injection	Fixed template-based emails; no free-form LLM input used for email body construction
API Key Exposure	.env file excluded via .gitignore; no hardcoded secrets in source code

Risk Area	Mitigation Applied
Data Privacy	Mock data only; no real customer financial information stored in the repository
Hallucination Risk	Template logic prevents AI-generated inaccuracies in financial figures or dates
Email Spoofing	Dry-run simulation mode; no real SMTP dispatch during testing phase

9. Limitations

- The system currently operates on static CSV data; there is no integration with live ERP or accounting systems such as SAP, Tally, or QuickBooks.
- Email dispatch is simulated; no real SMTP server is configured in this version.
- The tone escalation rules are hard-coded thresholds and do not adapt to client payment history or relationship context.
- The application does not support multi-user access or role-based permissions.
- All overdue day counts are taken directly from the CSV rather than calculated dynamically from the current date.

10. Future Improvements

Enhancement	Description
Real SMTP Integration	Connect to Gmail/Outlook API for actual email delivery
AI-Based Personalisation	Use LLMs to personalise email tone based on client relationship history
Live Data Source	Replace CSV with database or ERP API integration
Payment Prediction	ML model to predict likelihood of payment based on historical patterns
Analytics Dashboard	Visualise collection performance metrics over time
Multi-language Support	Generate emails in regional languages for diverse client bases
Dynamic Overdue Calc	Auto-calculate days_overdue from current date rather than manual CSV entry

11. Conclusion

The Finance Credit Follow-Up Email Agent successfully demonstrates how lightweight automation can reduce manual effort in enterprise finance operations. By combining structured data ingestion, rule-based intelligence, and a polished interactive UI, the project delivers a functional prototype that can be extended into a production-grade collections management tool.

The tone escalation framework is the system's primary innovation: it enforces a graduated communication strategy that balances client relationship preservation with firm collections enforcement. The audit logging feature adds operational transparency and compliance readiness.

This project demonstrates proficiency in Python application development, Streamlit-based UI design, business logic implementation, and awareness of security concerns relevant to financial data applications — fulfilling the objectives of the AI Enablement Internship Project Challenge.

Appendix A: Installation & Setup

Prerequisites

- Python 3.8 or higher
- pip package manager
- Git

Steps

1. Clone the repository: git clone <your-github-link>
2. Navigate to the project folder: cd finance-email-agent
3. Install dependencies: pip install -r requirements.txt
4. Run the application: python -m streamlit run app.py

The application will open in your default web browser at http://localhost:8501.